

MapaGlobalFinal

August 21, 2026

1 Mapa global en Coppeliasim

Este notebook tiene como fin construir un mapa local instantáneo y un mapa global acumulativo utilizando los 16 sensores ultrasónicos fijos del Pioneer P3DX.

1.1 1. Librerías

Importar únicamente las librerías necesarias para comunicación con Coppeliasim, operaciones matemáticas, almacenamiento y visualización.

```
[1]: import math
import csv
import pygame

from coppeliasim_zmqremoteapi_client import RemoteAPIClient
```

pygame 2.6.1 (SDL 2.28.4, Python 3.10.12)

Hello from the pygame community. <https://www.pygame.org/contribute.html>

1.2 2. Parámetros generales del sistema

```
[2]: # =====
# MOVIMIENTO DEL ROBOT
# =====

LINEAR_WHEEL_SPEED = 2.0
TURN_WHEEL_SPEED = 1.5

# =====
# MAPAS
# =====

LOCAL_RANGE = 5.0      # metros mostrados desde el robot
GLOBAL_RANGE = 10.0    # metros mostrados respecto al mundo

MAP_RESOLUTION = 0.05  # tamaño de celda en metros (5 cm)
```

```
# =====
# VENTANA
# =====

WINDOW_WIDTH = 1300
WINDOW_HEIGHT = 700

FPS = 60
```

1.3 3. Conexión con CoppeliaSim

Crear la conexión mediante la API remota ZMQ.

Se habilitará el modo `stepping` para controlar desde Python cada avance de la simulación.

```
[3]: client = RemoteAPIClient()

sim = client.require("sim")

# Permitir que Python controle cada paso de simulación.
sim.setStepping(True)

print("Conexión con CoppeliaSim establecida.")
```

Conexión con CoppeliaSim establecida.

1.4 4. Obtener los objetos del robot

Buscar los motores y los 16 sensores ultrasónicos dentro de la escena.

```
[4]: # Robot principal.
robot = sim.getObject("/PioneerP3DX")

# Motores.
left_motor = sim.getObject("/PioneerP3DX/leftMotor")
right_motor = sim.getObject("/PioneerP3DX/rightMotor")

# 16 sensores ultrasónicos.
sensors = [
    sim.getObject(f"/PioneerP3DX/ultrasonicSensor[{i}]",
    for i in range(16)
]

print("Robot:", robot)
print("Motor izquierdo:", left_motor)
print("Motor derecho:", right_motor)
print("Sensores encontrados:", len(sensors))
```

Robot: 102
Motor izquierdo: 107
Motor derecho: 104
Sensores encontrados: 16

1.5 5. Centro de rotación del robot

Utilizar como referencia local el punto medio entre las dos ruedas.

Si las posiciones de ambas ruedas respecto al chasis son:

$$P_L = (x_L, y_L, z_L)$$

y

$$P_R = (x_R, y_R, z_R),$$

entonces el centro se calculará como:

$$P_C = \left(\frac{x_L + x_R}{2}, \frac{y_L + y_R}{2}, \frac{z_L + z_R}{2} \right)$$

Este punto será el origen del mapa local.

```
[5]: left_pos = sim.getObjectPosition(left_motor, robot)
right_pos = sim.getObjectPosition(right_motor, robot)

rotation_center_robot = [
    (left_pos[0] + right_pos[0]) / 2.0,
    (left_pos[1] + right_pos[1]) / 2.0,
    (left_pos[2] + right_pos[2]) / 2.0,
]

print("Centro de rotación respecto al robot:")
print(rotation_center_robot)
```

Centro de rotación respecto al robot:

[0.044512439519166946, -8.866190910339355e-07, -0.041263267397880554]

1.6 6. Función para transformar un punto

CoppeliaSim devuelve matrices de transformación (3×4).

Aplicar una matriz de este tipo a un punto:

$$P = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

permitirá expresar una detección en otro sistema de coordenadas.

Esta función será utilizada tanto para el mapa local como para el global.

```
[6]: def transform_point(matrix, point):
```

```
    x = (
        matrix[0] * point[0]
        + matrix[1] * point[1]
        + matrix[2] * point[2]
        + matrix[3]
    )

    y = (
        matrix[4] * point[0]
        + matrix[5] * point[1]
        + matrix[6] * point[2]
        + matrix[7]
    )

    z = (
        matrix[8] * point[0]
        + matrix[9] * point[1]
        + matrix[10] * point[2]
        + matrix[11]
    )

    return x, y, z
```

1.7 7. Pose global del robot

Obtener la posición y orientación actual del Pioneer dentro del mundo.

El ángulo yaw corresponde a la rotación alrededor del eje Z y será utilizado para dibujar la orientación del robot en el mapa global.

```
[7]: def get_robot_pose():
```

```
    position = sim.getObjectPosition(robot, -1)
    orientation = sim.getObjectOrientation(robot, -1)

    x = position[0]
    y = position[1]
    z = position[2]
    yaw = orientation[2]

    return x, y, z, yaw
```

```
def get_rotation_center_world():
    '''
    Expresar el centro de rotación del robot en coordenadas globales.
    '''

    robot_to_world = sim.getObjectMatrix(robot, -1)

    return transform_point(
        robot_to_world,
        rotation_center_robot
    )
```

1.8 8. Control de los motores

Separar el control físico del robot de la lectura del teclado.

De esta manera, posteriormente que desémos controlar el robot con el joystick o un algoritmo autónomo por medio de ROS2, bastará con reemplazar esta función.

```
[8]: def set_wheel_velocities(left_velocity, right_velocity):
    '''
    Asignar velocidad angular a las dos ruedas.
    '''

    sim.setJointTargetVelocity(
        left_motor,
        float(left_velocity)
    )

    sim.setJointTargetVelocity(
        right_motor,
        float(right_velocity)
    )

def stop_robot():
    '''
    Detener completamente el Pioneer 3DX.
    '''

    set_wheel_velocities(0.0, 0.0)
```

1.9 9. Estructuras para almacenar el mapa

Crear las variables donde se conservarán los obstáculos detectados y la trayectoria recorrida.

`global_map` almacenará índices discretizados de celdas.

`global_points` conservará coordenadas métricas representativas de esas celdas.

trajectory almacenará el recorrido del centro de rotación del robot.

```
[9]: global_map = set()

global_points = {}

trajectory = []

current_command = "DETENIDO"
```

1.10 10. Discretización del mapa global

Los sensores pueden generar pequeñas variaciones incluso cuando observan el mismo objeto.

Para evitar guardar cientos de puntos casi idénticos, convertir las coordenadas globales a una cuadrícula.

Con resolución (r):

$$i_x = \text{round}\left(\frac{x}{r}\right)$$

$$i_y = \text{round}\left(\frac{y}{r}\right)$$

```
[10]: def discretize_global_point(x, y):
    '''
    Convertir una posición métrica a índices de celda.
    '''

    ix = round(x / MAP_RESOLUTION)
    iy = round(y / MAP_RESOLUTION)

    return ix, iy

def add_global_point(x, y, z):
    '''
    Añadir una detección al mapa global evitando duplicados
    dentro de una misma celda.
    '''

    cell = discretize_global_point(x, y)

    global_map.add(cell)

    global_points[cell] = (
        cell[0] * MAP_RESOLUTION,
        cell[1] * MAP_RESOLUTION,
```

```
)  
    z
```

1.11 11. Lectura de los 16 sensores

Cada sensor devuelve el punto detectado en su propio sistema de coordenadas.

Para construir el mapa local realizar:

$$P_R = {}^R T_{S_i} P_{S_i}$$

Después desplazar el origen al centro de rotación.

Para construir el mapa global realizar:

$$P_G = {}^G T_{S_i} P_{S_i}$$

Al utilizar la matriz real de cada sensor, no será necesario asignar manualmente los ángulos de los 16 sensores.

```
[11]: def read_sensors():  
    '''  
    Leer los 16 sensores ultrasónicos.  
  
    Returns  
    -----  
    list  
        Lista de diccionarios. Cada diccionario representa  
        una detección válida.  
    '''  
  
    detections = []  
  
    for sensor_index, sensor in enumerate(sensors):  
  
        (  
            result,  
            sensor_distance,  
            detected_point,  
            detected_object,  
            normal  
        ) = sim.readProximitySensor(sensor)  
  
        # result <= 0 significa que no existe detección.  
        if result <= 0:  
            continue  
  
        # =====
```

```

# 1. TRANSFORMACIÓN SENSOR -> ROBOT
# =====

sensor_to_robot = sim.getObjectMatrix(
    sensor,
    robot
)

x_robot, y_robot, z_robot = transform_point(
    sensor_to_robot,
    detected_point
)

# Cambiar el origen al centro de rotación.
x_local = x_robot - rotation_center_robot[0]
y_local = y_robot - rotation_center_robot[1]
z_local = z_robot - rotation_center_robot[2]

# =====
# 2. TRANSFORMACIÓN SENSOR -> MUNDO
# =====

sensor_to_world = sim.getObjectMatrix(
    sensor,
    -1
)

x_global, y_global, z_global = transform_point(
    sensor_to_world,
    detected_point
)

# Añadir el obstáculo al mapa acumulativo.
add_global_point(
    x_global,
    y_global,
    z_global
)

# =====
# 3. GUARDAR INFORMACIÓN DE LA DETECCIÓN
# =====

detections.append({
    "sensor": sensor_index,

```

```

        "x_local": x_local,
        "y_local": y_local,
        "z_local": z_local,

        "x_global": x_global,
        "y_global": y_global,
        "z_global": z_global,

        "distance_sensor": sensor_distance,

        "distance_robot": math.hypot(
            x_local,
            y_local
        ),

        "angle_local_deg": math.degrees(
            math.atan2(
                y_local,
                x_local
            )
        ),

        "object_handle": detected_object
    })

    return detections

```

1.12 12. Inicialización de Pygame

```

[12]: pygame.init()

pygame.display.set_caption(
    "Pioneer P3DX | Mapa local + mapa global | W A S D"
)

screen = pygame.display.set_mode(
    (WINDOW_WIDTH, WINDOW_HEIGHT)
)

font = pygame.font.SysFont(
    "DejaVu Sans",
    17
)

small_font = pygame.font.SysFont(
    "DejaVu Sans",

```

```

    14
)

clock = pygame.time.Clock()

print("Ventana de Pygame preparada.")

```

Ventana de Pygame preparada.

1.13 13. Conversión de metros a píxeles

Los sensores trabajan en metros, pero Pygame dibuja en píxeles.

Esta función realizará el cambio de escala entre ambos sistemas.

```

[13]: def metric_to_panel(
    x,
    y,
    x_min,
    x_max,
    y_min,
    y_max,
    panel_left,
    panel_top,
    panel_width,
    panel_height
):
    nx = (x - x_min) / (x_max - x_min)
    ny = (y - y_min) / (y_max - y_min)

    px = int(
        panel_left
        + nx * panel_width
    )

    py = int(
        panel_top
        + (1.0 - ny) * panel_height
    )

    return px, py

```

1.14 14. Cuadrícula de los mapas

Dibujar una cuadrícula común para los paneles local y global.

Los ejes centrales representan ($X=0$) y ($Y=0$) en el sistema de referencia mostrado.

```

[14]: def draw_panel_grid(
    rect,
    coordinate_range,
    title
):
    '''
    Dibujar fondo, cuadrícula, ejes y título de un mapa.
    '''

    left, top, width, height = rect

    pygame.draw.rect(
        screen,
        (20, 20, 20),
        rect
    )

    pygame.draw.rect(
        screen,
        (100, 100, 100),
        rect,
        1
    )

    r = coordinate_range

    meter_min = math.floor(-r)
    meter_max = math.ceil(r)

    for meter in range(
        meter_min,
        meter_max + 1
    ):

        px, _ = metric_to_panel(
            meter,
            0,
            -r,
            r,
            -r,
            r,
            left,
            top,
            width,
            height
        )

```

```

_, py = metric_to_panel(
    0,
    meter,
    -r,
    r,
    -r,
    r,
    left,
    top,
    width,
    height
)

pygame.draw.line(
    screen,
    (45, 45, 45),
    (px, top),
    (px, top + height),
    1
)

pygame.draw.line(
    screen,
    (45, 45, 45),
    (left, py),
    (left + width, py),
    1
)

# Ejes principales.
px0, py0 = metric_to_panel(
    0,
    0,
    -r,
    r,
    -r,
    r,
    left,
    top,
    width,
    height
)

pygame.draw.line(
    screen,
    (100, 100, 100),
    (left, py0),

```

```

        (left + width, py0),
        2
    )

    pygame.draw.line(
        screen,
        (100, 100, 100),
        (px0, top),
        (px0, top + height),
        2
    )

    title_surface = font.render(
        title,
        True,
        (240, 240, 240)
    )

    screen.blit(
        title_surface,
        (left + 8, top + 8)
    )

```

1.15 15. Dibujo del mapa local

En este mapa el robot permanecerá siempre en el origen.

Los puntos detectados cambiarán de posición alrededor del robot según las lecturas instantáneas de los 16 sensores.

```

[15]: def draw_local_map(
        detections,
        rect
    ):
        '''
        Dibujar detecciones respecto al centro de rotación.
        '''

        left, top, width, height = rect
        r = LOCAL_RANGE

        draw_panel_grid(
            rect,
            r,
            "MAPA LOCAL - centro de rotación"
        )

        # Centro del robot.

```

```

cx, cy = metric_to_panel(
    0,
    0,
    -r,
    r,
    -r,
    r,
    left,
    top,
    width,
    height
)

# Símbolo triangular del robot apuntando a +X.
robot_shape = [
    (cx + 24, cy),
    (cx - 16, cy - 16),
    (cx - 16, cy + 16)
]

pygame.draw.polygon(
    screen,
    (70, 160, 240),
    robot_shape
)

# Dibujar cada detección.
for detection in detections:

    px, py = metric_to_panel(
        detection["x_local"],
        detection["y_local"],
        -r,
        r,
        -r,
        r,
        left,
        top,
        width,
        height
    )

    # Línea robot -> obstáculo.
    pygame.draw.line(
        screen,
        (90, 90, 90),
        (cx, cy),

```

```

        (px, py),
        1
    )

    # Punto de obstáculo.
    pygame.draw.circle(
        screen,
        (235, 90, 90),
        (px, py),
        6
    )

    # Identificador del sensor.
    label = small_font.render(
        f"S{detection['sensor']}",
        True,
        (245, 245, 245)
    )

    screen.blit(
        label,
        (px + 7, py - 7)
    )

```

1.16 16. Dibujo del mapa global

El mapa global conservará todas las celdas ocupadas detectadas durante el recorrido.

A diferencia del mapa local, el robot se moverá dentro de este sistema de referencia.

```

[16]: def draw_global_map(rect):
    '''
    Dibujar obstáculos acumulados, trayectoria y pose del robot.
    '''

    left, top, width, height = rect
    r = GLOBAL_RANGE

    draw_panel_grid(
        rect,
        r,
        "MAPA GLOBAL - coordenadas del mundo"
    )

    # =====
    # OBSTÁCULOS ACUMULADOS
    # =====

```

```

for x, y, z in global_points.values():

    if not (
        -r <= x <= r
        and
        -r <= y <= r
    ):
        continue

    px, py = metric_to_panel(
        x,
        y,
        -r,
        r,
        -r,
        r,
        left,
        top,
        width,
        height
    )

    pygame.draw.circle(
        screen,
        (235, 90, 90),
        (px, py),
        2
    )

# =====
# TRAYECTORIA
# =====

if len(trajjectory) >= 2:

    trajectory_pixels = []

    for x, y in trajectory:

        if (
            -r <= x <= r
            and
            -r <= y <= r
        ):

            trajectory_pixels.append(

```

```

        metric_to_panel(
            x,
            y,
            -r,
            r,
            -r,
            r,
            left,
            top,
            width,
            height
        )
    )

    if len(trjectory_pixels) >= 2:

        pygame.draw.lines(
            screen,
            (120, 190, 250),
            False,
            trajectory_pixels,
            2
        )

# =====
# POSE ACTUAL DEL ROBOT
# =====

x_center, y_center, z_center = get_rotation_center_world()

_, _, _, yaw = get_robot_pose()

cx, cy = metric_to_panel(
    x_center,
    y_center,
    -r,
    r,
    -r,
    r,
    left,
    top,
    width,
    height
)

arrow_length = 28

```

```

end_x = cx + int(
    arrow_length
    * math.cos(yaw)
)

end_y = cy - int(
    arrow_length
    * math.sin(yaw)
)

pygame.draw.circle(
    screen,
    (70, 160, 240),
    (cx, cy),
    7
)

pygame.draw.line(
    screen,
    (70, 160, 240),
    (cx, cy),
    (end_x, end_y),
    4
)

```

1.17 17. Información de estado

Mostrar en la parte superior la información más importante de cada ciclo: movimiento, número de sensores detectando, pose y cantidad de celdas ocupadas.

```

[17]: def draw_status(
    detections,
    command
):
    '''
    Mostrar información de ejecución.
    '''

    x_center, y_center, z_center = get_rotation_center_world()

    _, _, _, yaw = get_robot_pose()

    lines = [
        f"Comando: {command}",
        f"Detecciones actuales: {len(detections)}/16",
        f"Celdas ocupadas acumuladas: {len(global_map)}",
    ]

```

```

(
    f"Centro global: "
    f"X={x_center:+.2f} m  "
    f"Y={y_center:+.2f} m"
),
(
    f"Yaw: "
    f"{math.degrees(yaw):+.1f} grados"
),
(
    f"Resolución mapa: "
    f"{MAP_RESOLUTION:.3f} m/celda"
),
(
    "W/A/S/D mover | "
    "ESPACIO detener | "
    "R borrar mapa | "
    "ESC salir"
)
]

y = 10

for line in lines:

    surface = small_font.render(
        line,
        True,
        (235, 235, 235)
    )

    screen.blit(
        surface,
        (10, y)
    )

    y += 20

```

1.18 18. Composición de la ventana

Combinar el mapa local, el mapa global y el panel de estado dentro de una sola ventana.

```

[18]: def draw_interface(
        detections,
        command
    ):
        """

```

Dibujar la interfaz completa.
'''

```
screen.fill(
    (12, 12, 12)
)

margin = 20
status_height = 150
gap = 20

panel_top = status_height

panel_height = (
    WINDOW_HEIGHT
    - panel_top
    - margin
)

panel_width = (
    WINDOW_WIDTH
    - 2 * margin
    - gap
) // 2

# Panel izquierdo.
local_rect = (
    margin,
    panel_top,
    panel_width,
    panel_height
)

# Panel derecho.
global_rect = (
    margin
    + panel_width
    + gap,
    panel_top,
    panel_width,
    panel_height
)

draw_local_map(
```

```

        detections,
        local_rect
    )

    draw_global_map(
        global_rect
    )

    draw_status(
        detections,
        command
    )

    pygame.display.flip()

```

1.19 19. Lectura del teclado

Mantener la lógica de teclado en una función independiente.

```

[19]: def process_keyboard(running):
    '''
    Procesar eventos y teclas W/A/S/D.

    Returns
    -----
    running : bool
        Indicar si continuar la ejecución.

    command : str
        Texto del movimiento actual.
    '''

    # =====
    # EVENTOS
    # =====

    for event in pygame.event.get():

        if event.type == pygame.QUIT:
            running = False

        elif event.type == pygame.KEYDOWN:

            if event.key == pygame.K_ESCAPE:
                running = False

            elif event.key == pygame.K_SPACE:

```

```

        stop_robot()

    elif event.key == pygame.K_r:

        global_map.clear()
        global_points.clear()
        trajectory.clear()

        print("Mapa global borrado.")

# =====
# ESTADO DEL TECLADO
# =====

keys = pygame.key.get_pressed()

if keys[pygame.K_w]:

    set_wheel_velocities(
        LINEAR_WHEEL_SPEED,
        LINEAR_WHEEL_SPEED
    )

    command = "AVANZAR"

elif keys[pygame.K_s]:

    set_wheel_velocities(
        -LINEAR_WHEEL_SPEED,
        -LINEAR_WHEEL_SPEED
    )

    command = "RETROCEDER"

elif keys[pygame.K_a]:

    set_wheel_velocities(
        -TURN_WHEEL_SPEED,
        TURN_WHEEL_SPEED
    )

    command = "GIRO IZQUIERDA"

```

```

elif keys[pygame.K_d]:

    set_wheel_velocities(
        TURN_WHEEL_SPEED,
        -TURN_WHEEL_SPEED
    )

    command = "GIRO DERECHA"

else:

    stop_robot()

    command = "DETENIDO"

return running, command

```

1.20 20. Registro de trayectoria

Guardar un nuevo punto sólo cuando el centro del robot haya avanzado una distancia mínima. Esto evitará almacenar muchos puntos prácticamente iguales cuando el robot esté detenido.

```

[20]: def update_trajectory(
        minimum_distance=0.02
    ):
        '''
        Añadir la posición del robot a la trayectoria
        únicamente si se ha desplazado lo suficiente.
        '''

        x, y, z = get_rotation_center_world()

        if not trajectory:

            trajectory.append(
                (x, y)
            )

            return

        previous_x, previous_y = trajectory[-1]

        distance = math.hypot(

```

```

        x - previous_x,
        y - previous_y
    )

    if distance >= minimum_distance:

        trajectory.append(
            (x, y)
        )

```

1.21 21. Exportación de resultados

Guardar el mapa global y la trayectoria en archivos CSV.

Esta sección puede modificarse posteriormente para almacenar también tiempo, número de sensor, probabilidad de ocupación u otros datos.

```

[21]: def save_results(
    map_filename="mapa_global_16_sensores.csv",
    trajectory_filename="trayectoria_robot.csv"
):
    '''
    Guardar mapa global y trayectoria en archivos CSV.
    '''

    # =====
    # MAPA GLOBAL
    # =====

    with open(
        map_filename,
        "w",
        newline="",
        encoding="utf-8"
    ) as file:

        writer = csv.writer(file)

        writer.writerow([
            "x_global",
            "y_global",
            "z_global"
        ])

        for x, y, z in global_points.values():

            writer.writerow([
                x,

```

```

        y,
        z
    ])

# =====
# TRAYECTORIA
# =====

with open(
    trajectory_filename,
    "w",
    newline="",
    encoding="utf-8"
) as file:

    writer = csv.writer(file)

    writer.writerow([
        "x_robot",
        "y_robot"
    ])

    for x, y in trajectory:

        writer.writerow([
            x,
            y
        ])

print(
    "Mapa guardado en:",
    map_filename
)

print(
    "Trayectoria guardada en:",
    trajectory_filename
)

```

1.22 22. Inicio de la simulación

Iniciar CoppeliaSim y realizar algunos pasos iniciales para permitir que los sensores se estabilicen antes de comenzar el recorrido.

```
[22]: sim.startSimulation()

# Avanzar algunos pasos antes de iniciar las lecturas.
for _ in range(5):
    sim.step()

print("Simulación iniciada.")
```

Simulación iniciada.

1.23 23. Ciclo principal

Esta celda integra todos los bloques definidos anteriormente.

```
[23]: running = True
current_command = "DETENIDO"

try:

    while running:

        # -----
        # 1. CONTROL DEL ROBOT
        # -----

        running, current_command = process_keyboard(
            running
        )

        # -----
        # 2. AVANZAR LA SIMULACIÓN
        # -----

        sim.step()

        # -----
        # 3. LEER LOS 16 SENSORES
        # -----

        detections = read_sensors()

        # -----
        # 4. ACTUALIZAR TRAYECTORIA GLOBAL
        # -----
```

```

    update_trajectory()

    # -----
    # 5. DIBUJAR LA INTERFAZ
    # -----

    draw_interface(
        detections,
        current_command
    )

    # -----
    # 6. LIMITAR FRECUENCIA DE LA VENTANA
    # -----

    clock.tick(FPS)

finally:

    # Detener físicamente el robot.
    stop_robot()

    # Detener la simulación.
    try:
        sim.step()
        sim.stopSimulation()
    except Exception:
        pass

    # Cerrar Pygame.
    pygame.quit()

    # Guardar resultados.
    save_results()

```

Mapa guardado en: mapa_global_16_sensores.csv

Trayectoria guardada en: trayectoria_robot.csv

1.24 24. ¿Cómo se construye el mapa global?

Cada sensor (S_i) detecta un punto en su propio sistema de referencia:

$$P_{S_i} = \begin{bmatrix} x_s \\ y_s \\ z_s \end{bmatrix}$$

Para representar ese punto respecto al mundo obtener la matriz:

```
sensor_to_world = sim.getObjectMatrix(sensor, -1)
```

y calcular:

$$P_G = {}^G T_{S_i} P_{S_i}$$

La matriz proporcionada por CoppeliaSim ya incorporar:

- posición del sensor sobre el robot;
- orientación del sensor;
- posición global actual del robot;
- orientación global actual del robot.

Por ello los 16 sensores pueden estar colocados en diferentes posiciones y direcciones sin tener que escribir manualmente sus ángulos.

1.25 27. Siguiendo mejora: Occupancy Grid

El mapa implementado hasta aquí representa **puntos ocupados acumulados**.

Para crear un verdadero mapa de ocupación será necesario aplicar **ray casting** entre cada sensor y su obstáculo.

Así poder clasificar cada celda como:

- desconocida;
- libre;
- ocupada.

Esta estructura será posteriormente más adecuada para algoritmos de planificación de trayectorias como A*.

1.26 28. Conclusiones

Se logró implementar un sistema de mapeo global para el robot Pioneer. A partir de las mediciones obtenidas por cada sensor, fue posible determinar la posición de los obstáculos respecto al centro de rotación del robot y posteriormente transformar estas coordenadas locales al sistema de referencia global del entorno.

El uso de las matrices de transformación proporcionadas por CoppeliaSim permitió considerar automáticamente la posición y orientación de cada sensor, así como los cambios de posición y orientación del robot durante su desplazamiento. De esta manera, los obstáculos detectados pudieron conservar su ubicación dentro del mapa global mientras el robot recorría el escenario.

Asimismo, la discretización de las coordenadas permitió reducir la acumulación de mediciones repetidas sobre un mismo obstáculo y generar una representación más organizada del entorno. La visualización simultánea del mapa local y global facilitó observar la diferencia entre ambos sistemas de referencia y comprender el proceso de transformación utilizado para realizar el mapeo.

Finalmente, la implementación desarrollada establece una base para continuar con sistemas de navegación más completos. Como trabajo posterior, el mapa de puntos ocupados puede evolucionar

hacia un mapa de ocupación mediante técnicas de ray casting, permitiendo representar espacios libres, ocupados y desconocidos para posteriormente implementar algoritmos de planificación de trayectorias y navegación autónoma.